

Introducción teórico-práctica al Aprendizaje por Refuerzo

Clase 2 — Introducción

Profesor: Matias Bilkis



UBA
Universidad de Buenos Aires



¿Armaron grupos?

¿Están todxs en CAMPUS?

Dudas puntuales organización → matiasbilkis@gmail.com

¿Hicieron tarea?

¿Qué vimos la Clase 1?

Elementos de RL



Entorno

- Se encuentra en un “estado” S_t
- Dada una acción del agente A_t revela R_t
- Y además cambia su estado
- Las reglas (aleatorias!) están dadas por
$$p_E(s', r|h = H_t) = p_E(s' = S_t, r = R_t|h = H_t)$$



Agente

- Observa el estado del entorno S_{t+1}
- Elige acciones de acuerdo a una política π
- Busca maximizar el **retorno** (suma rewards)

El objetivo de RL

Obtener el **máximo retorno** dentro de un episodio

$$G_t = R_t + R_{t+1} + \dots R_L$$

¿Cómo venimos?

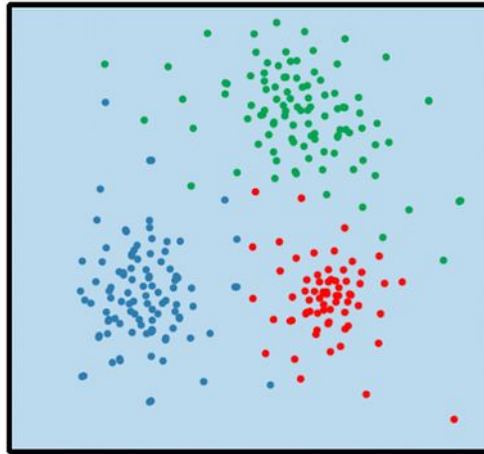
¿Les parece que está bien formulado esto?

$v(s) = \mathbb{E}[G_t | s = S_t]$ *¿De qué “objetos” debería depender este valor esperado? (pista = p^2)*

RL como un marco importantísimo

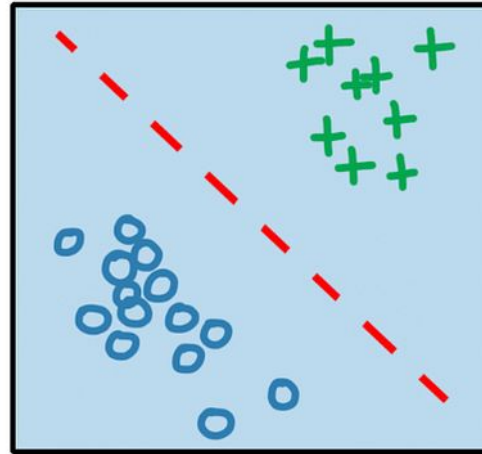
machine learning

unsupervised
learning



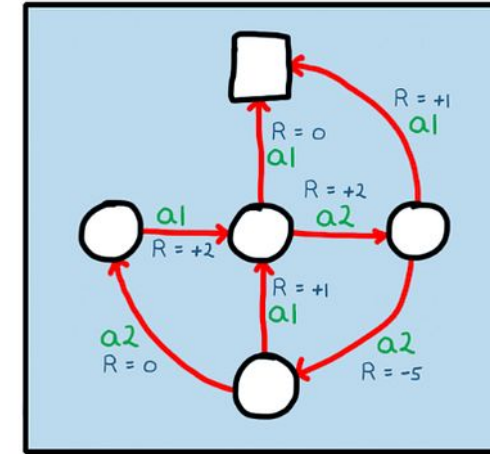
*El Gólem mira el piso y
descubre
“patrones”/”etiquetas”
(si está roñoso, o si está limpio)*

supervised
learning



*El rabino le
muestra fotos
“limpio”/”sucio”*

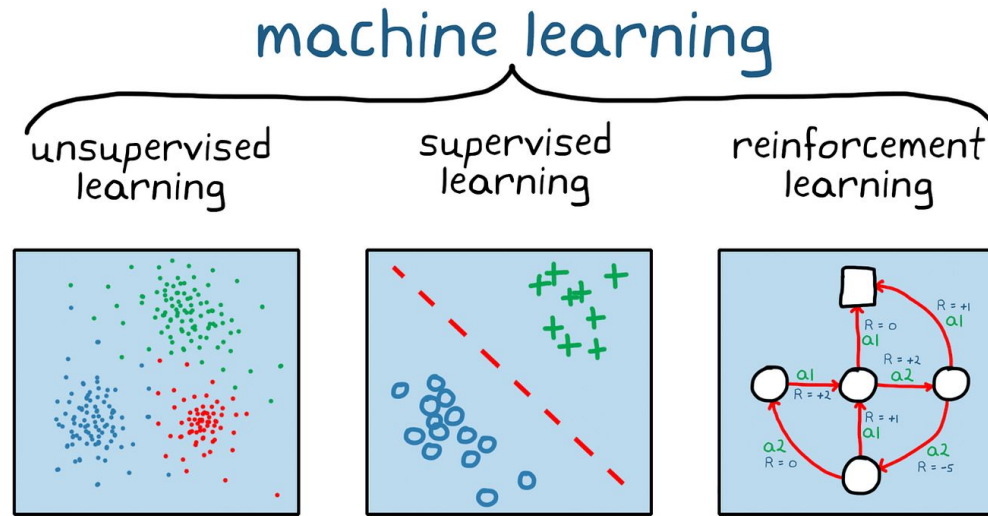
reinforcement
learning



*El rabino lo guía
con premios*



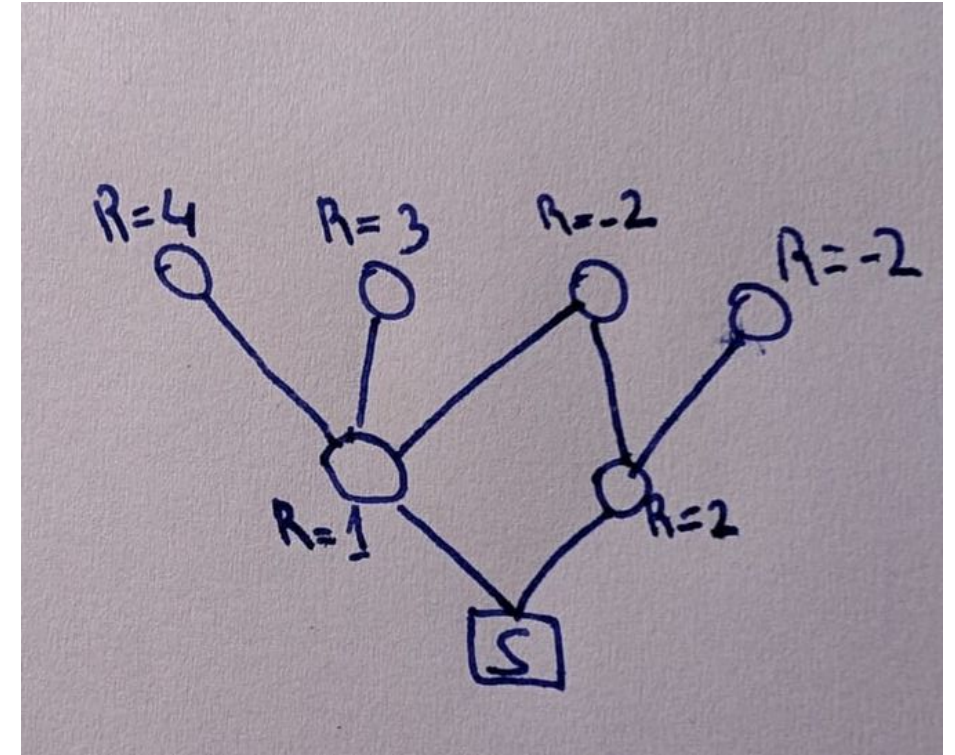
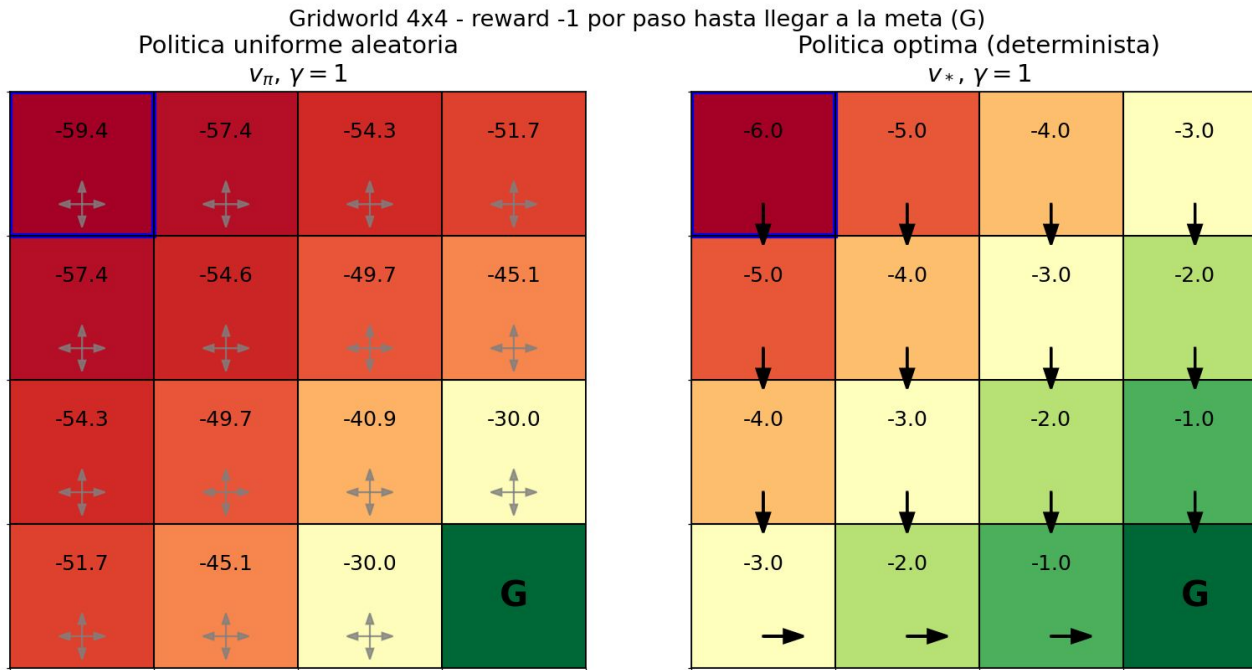
La diferencia con supervised..



En (SL) te dicen qué acción ejecutar.

Pero en (RL) tenemos que *evaluarla* (para ver si rinde!)

Evaluar en RL

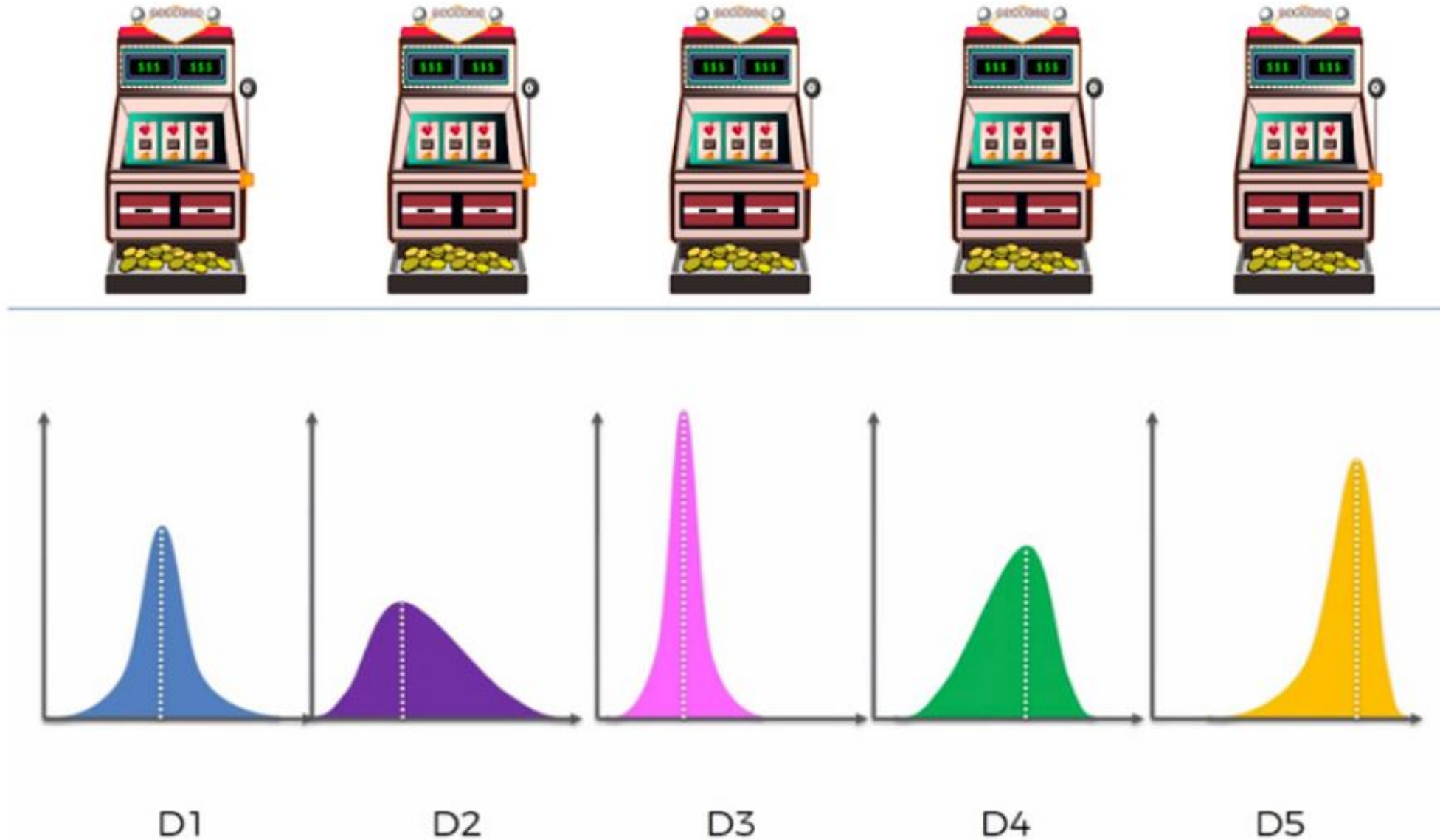


En (SL) te dicen qué acción ejecutar.

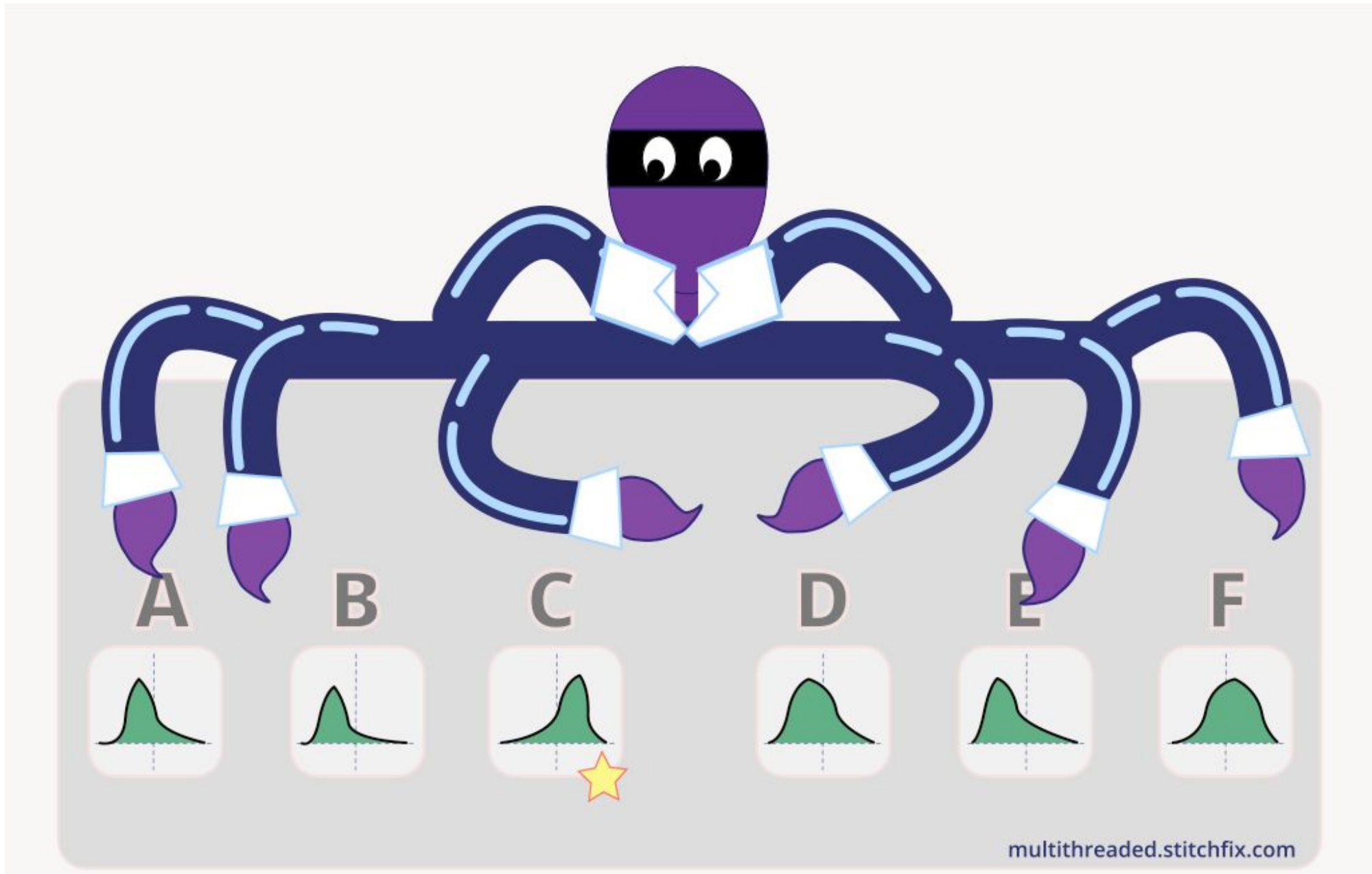
Pero en (RL) tenemos que *evaluarla* (para ver si rinde!)

Problema de los bandidos

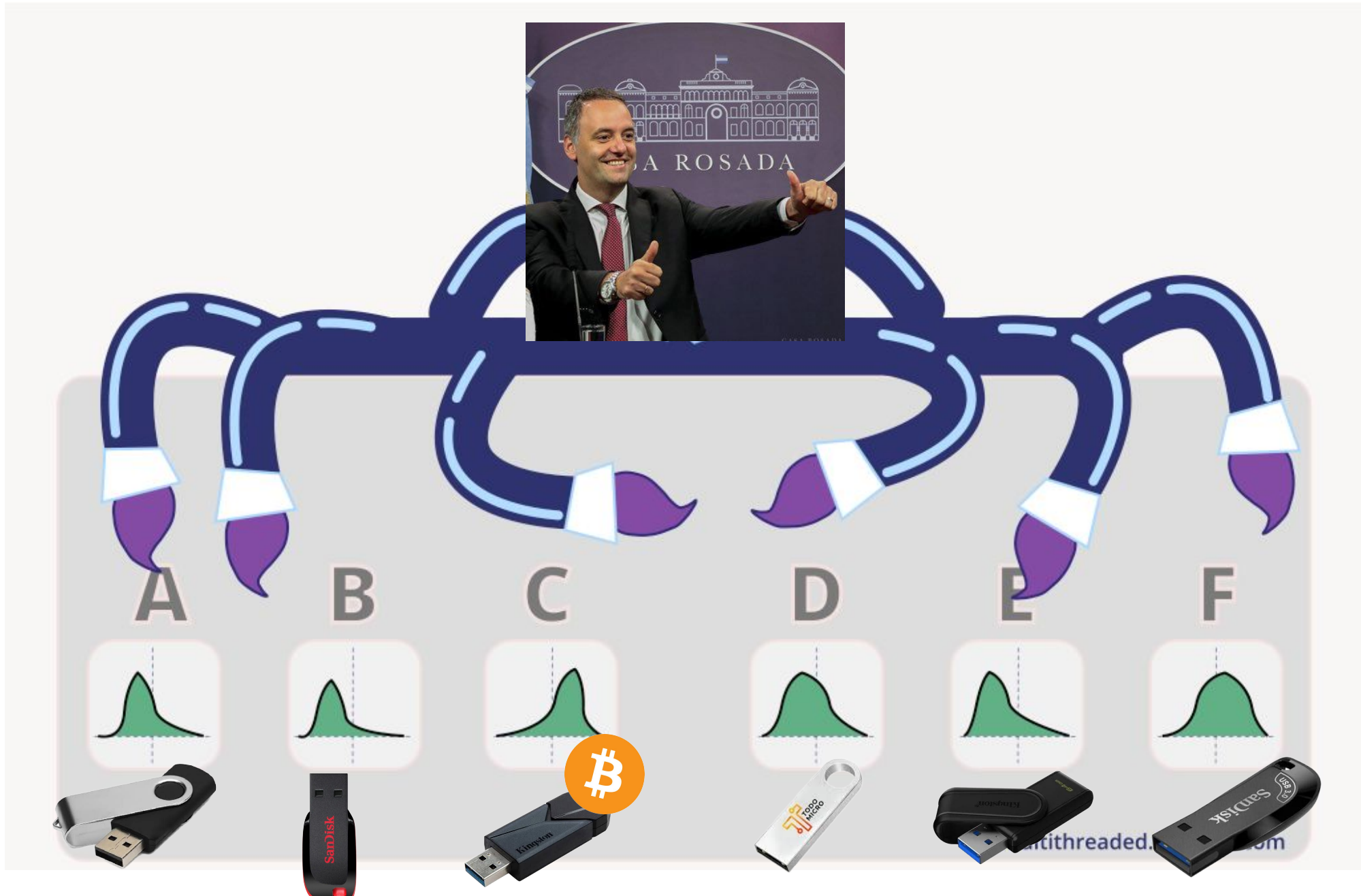
Problema de los bandidos



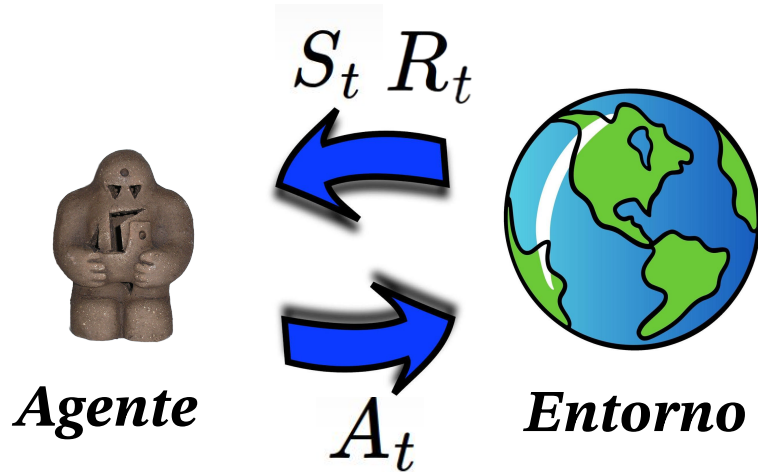
Problema de los bandidos



Problema de los bandidos



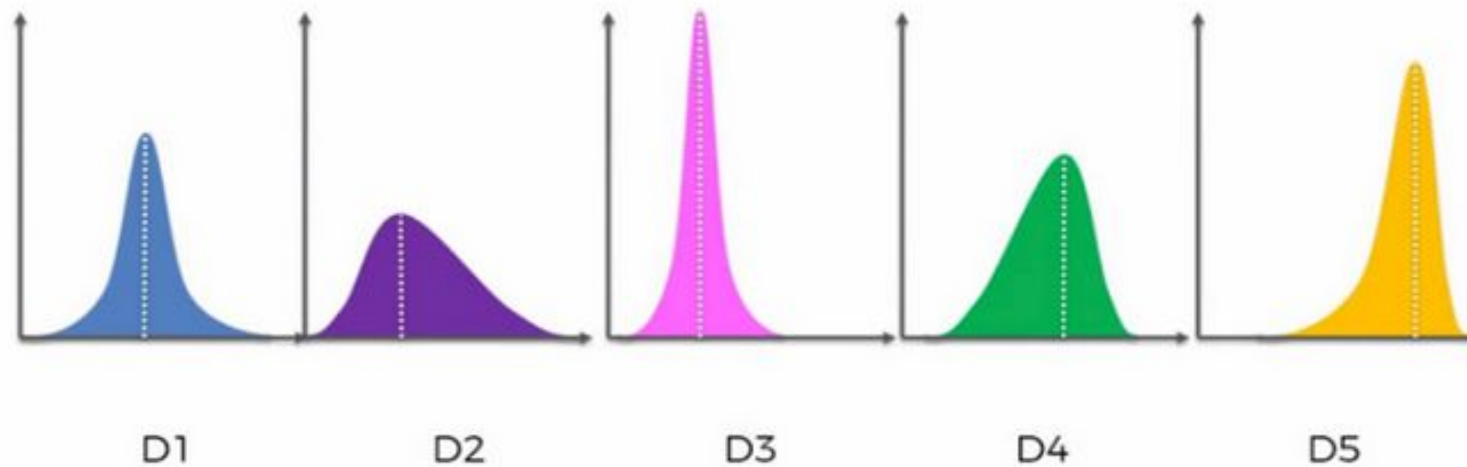
Bandits en el contexto de ayer



- *La acción que elijo solamente repectute en el próximo premio y termina el episodio*
- *No hay “estados” del entorno / agente (1 solo paso)*
- *¿¿Qué acción elijo??*

Problema de los bandidos

Tenemos k distribuciones de probabilidad diferentes.

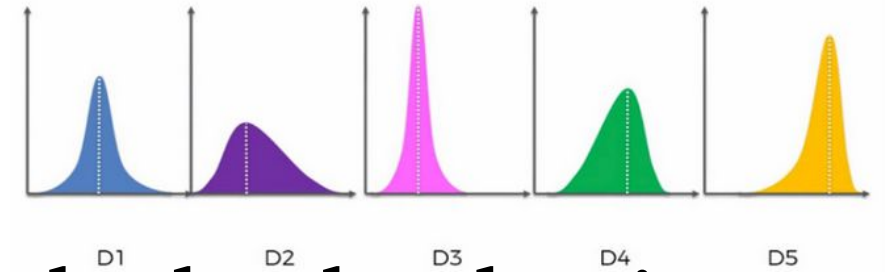


- Cada $p.d.f.$ nos da un reward si la sampleamos.

$$R_t \sim p(r|a = A_t)$$

- El agente desconoce $p(r|a = A_t)$

Definiciones



Objetivo: maximizar la suma de rewards desde el primer episodio

$$\bar{R} = \sum_i R_i$$

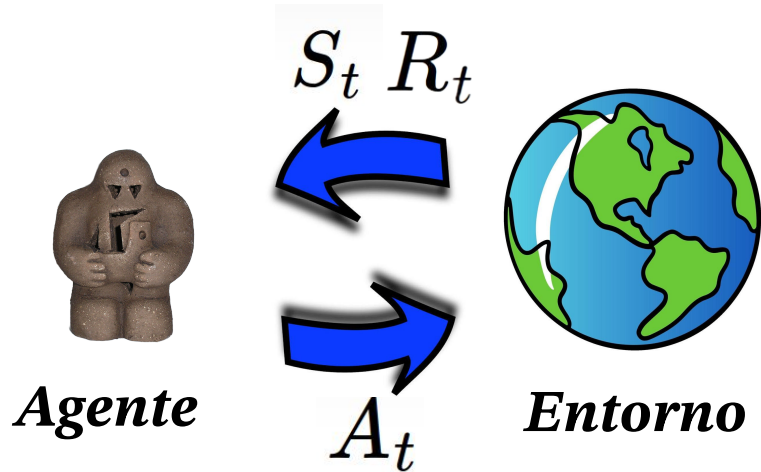
*Obvio que nos interesa
“ganar más plata”*



[\[Cómo ganar plata, OF\]](#)

Observación: $\bar{R}$ es una variable aleatoria...
¿de qué depende?

Bandits en el contexto de ayer

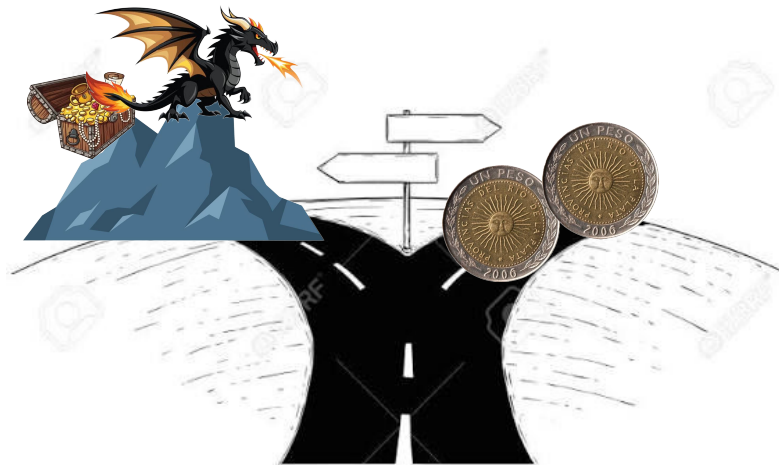


- ¿¿Qué acción elijo??

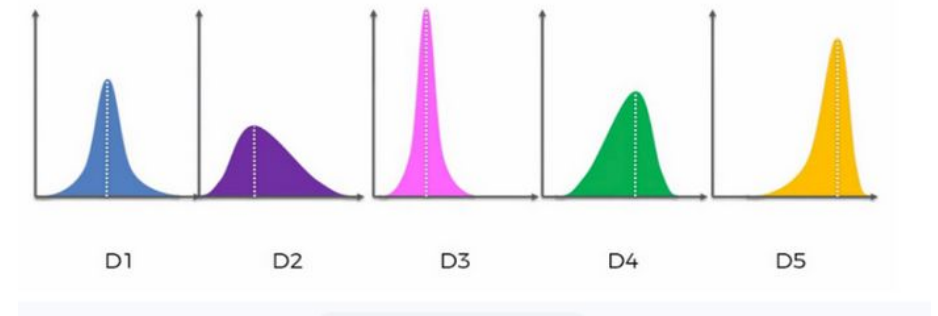
$$\bar{R} = \sum_i R_i$$

- Objetivo: acumular (sin horizonte)

- Tenemos que hacer un balance entre “ganancias” al corto plazo, y “potenciales” ganancias al largo plazo



Definiciones



Def. valor de la acción

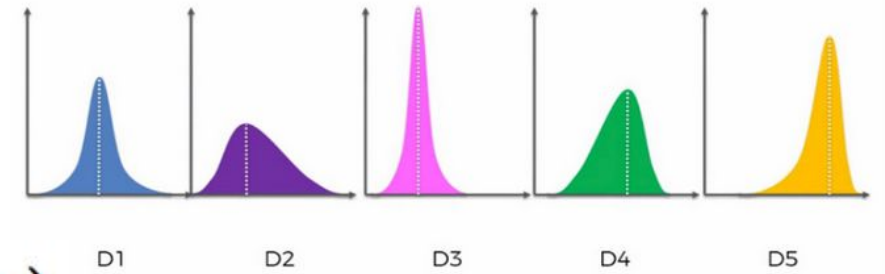
$$q(a) = \mathbb{E}[R_t | A_t = a]$$

- No depende de $\pi(A_t | S_t)$ porque no hay estados!
- Lo llamamos diferente que el Sutton! (sin el “*”)

Def. estimador del valor de la acción (*inicializamos en 0*)

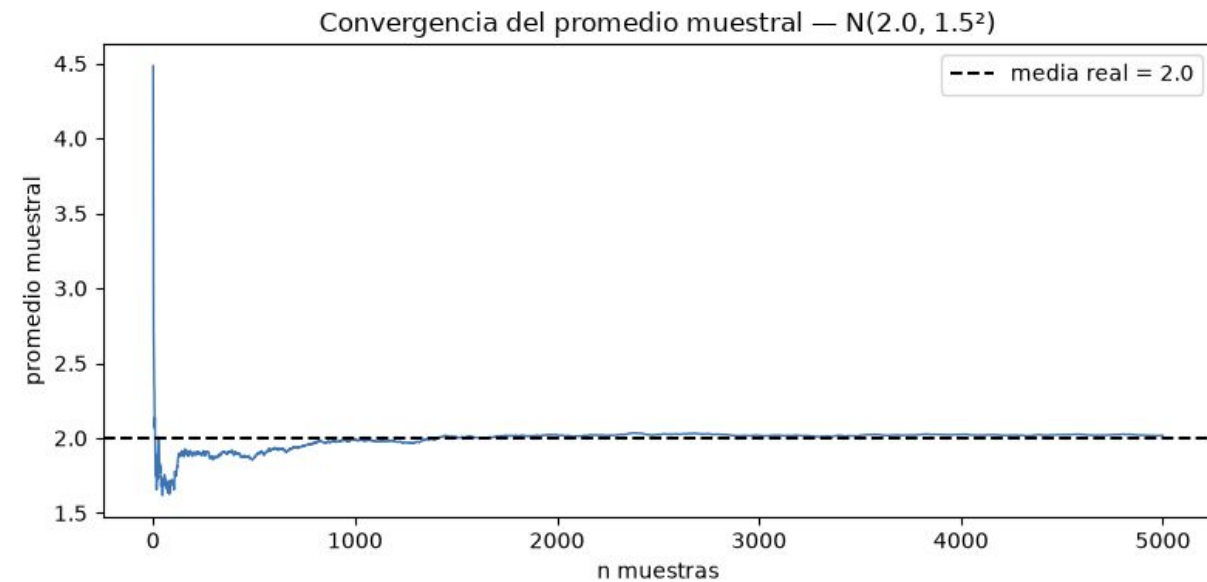
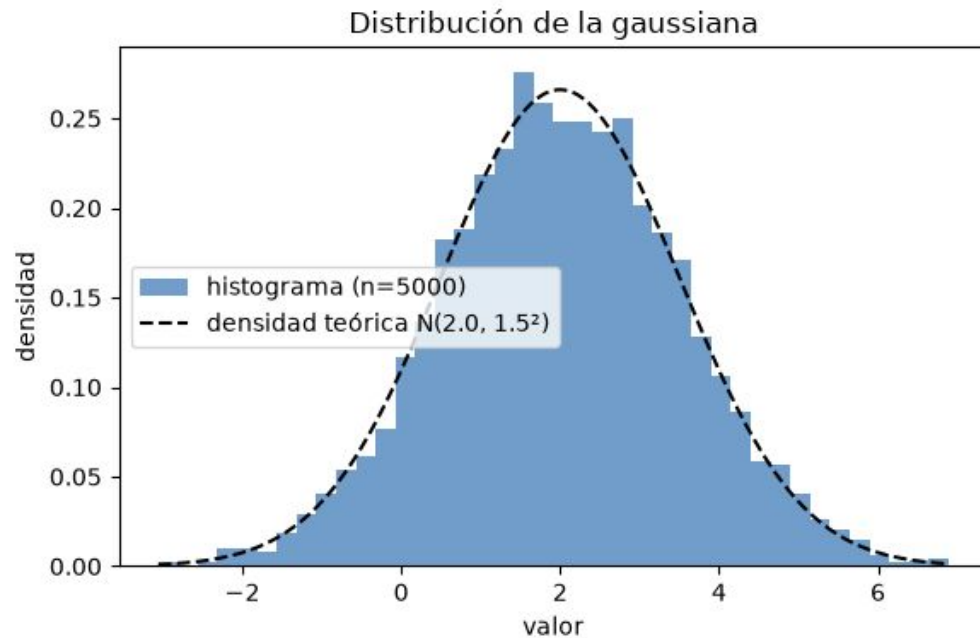
$$\hat{q}(a) = \frac{\sum_{i=1}^{t-1} R_i \mathbb{I}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{I}_{A_i=a}}$$

Observaciones

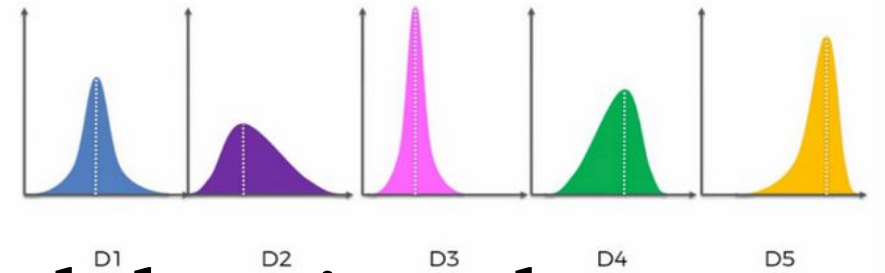


Lógicamente, queremos que $\hat{q}(a) \simeq q(a)$

Y por [\[Weak law of large numbers\]](#) en efecto converge (*rate?*)



Greedy actions



Imaginemos que estamos muy seguros del estimador...

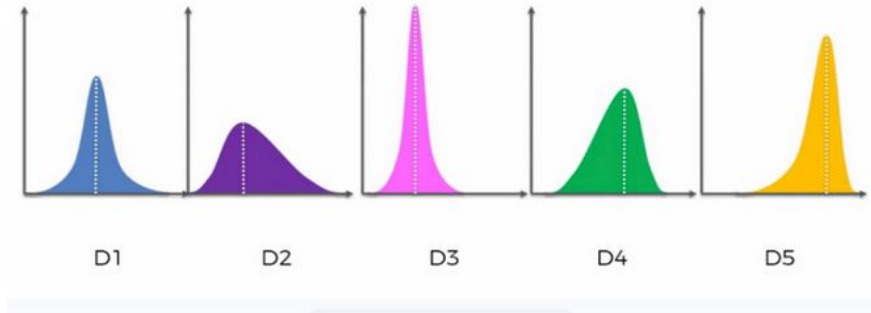
$$A_t = \underset{a}{\operatorname{argmax}} \hat{q}_t(a)$$

Greedy = “codiciosa / comilona”

ϵ -greedy ¿Pero si queremos explorar?

$$A_t = \begin{cases} \underset{a}{\operatorname{argmax}} \hat{q}_t(a) & \text{con prob} = 1 - \epsilon \\ \text{random} & \text{con prob} \epsilon \end{cases}$$

Casos para el ep-greedy



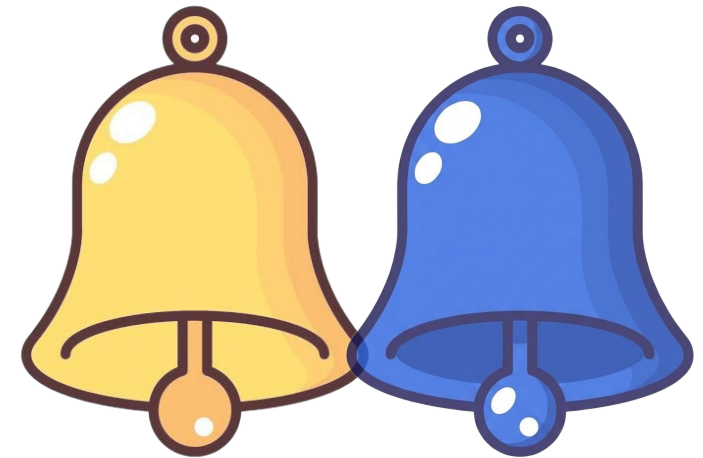
$\epsilon=1$ Uniformemente aleatoria

$\epsilon=0.5$ (...)

$\epsilon=0$ greedy



VS.



Pregunta: ¿en qué escenarios se imaginan que sería mejor tener una mayor / “mejor” exploración (epsilon más grande).

Implementación incremental

- Quizá esto sea obvio, pero es importante

$$\hat{q}_{n+1}(a) = \frac{R_1 + \dots + R_n}{n}$$

$$\begin{aligned}\hat{q}_{n+1}(a) &= \frac{R_1 + \dots + R_n}{n} \\ &= \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} [R_n + (n-1)\hat{q}_n(a)] \\ &= \hat{q}_n + \left(\frac{R_n - \hat{q}_n}{n} \right)\end{aligned}$$

cambio de notación temporal...

...asumimos a se prueba n veces, obteniendo $(R_1, \dots, R_n)$

$$\hat{q}_{n+1}(a) = \hat{q}_n(a) + \frac{1}{n} \left(R_n - \hat{q}_n(a) \right)$$

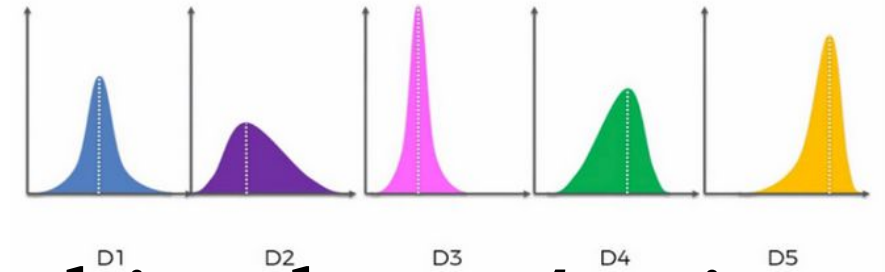
Estructura importante

$$\hat{q}_{\text{new}} = \hat{q}_{\text{old}} + \alpha_{\text{step}}[R_{\text{experience}} - \hat{q}_{\text{old}}]$$

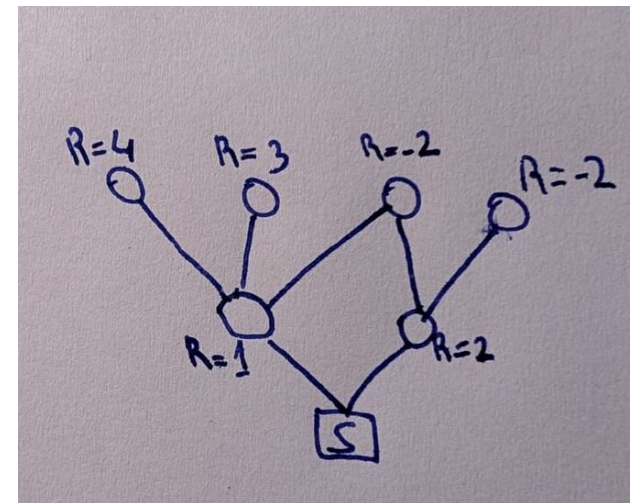
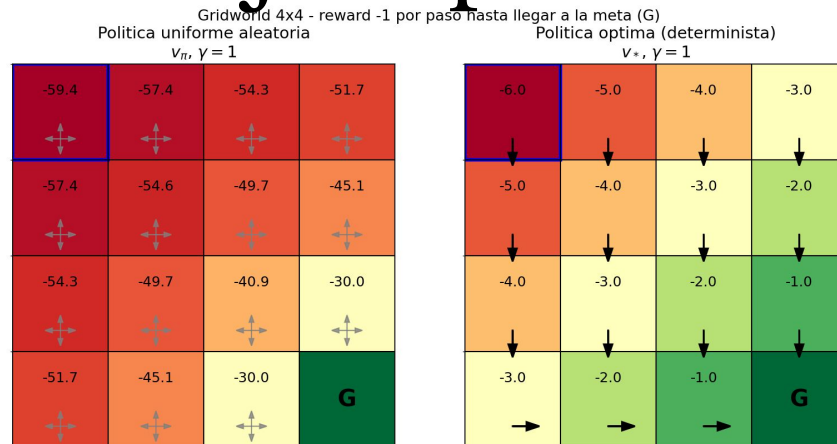
- En lugar de usar $1/n$, podemos dejar un α fijo
- Esto resulta en un promedio pesado, y los pesos son más chiquitos *cuanto más pasado*

$$\hat{q}_{n+1}(a) = (1 - \alpha)^n \hat{q}_1 + \sum_i \alpha (1 - \alpha)^{n-i} R_i$$

No estacionarios



- Las pdfs de los rewards cambian (cambian las máquinas..)
- En MDPs, al cambiar la policy, la distribución de rewards puede cambiar, incluso si “no cambian las máquinas!”
- Ejemplo: elije ir por un camiono distinto



Memento ejemplito

Nos damos 2' para pensar
cómo programar esto

Ejemplito

¿Cómo haríamos para programar y probar esta idea en un “entorno” de juguete?

- Definir los arms (*pdfs*)
- Definir el agente (la policy)
- Tener un loop que simule la interacción *Agente* <> *Arms*
- Evaluar cómo salió
- Juntar estadística

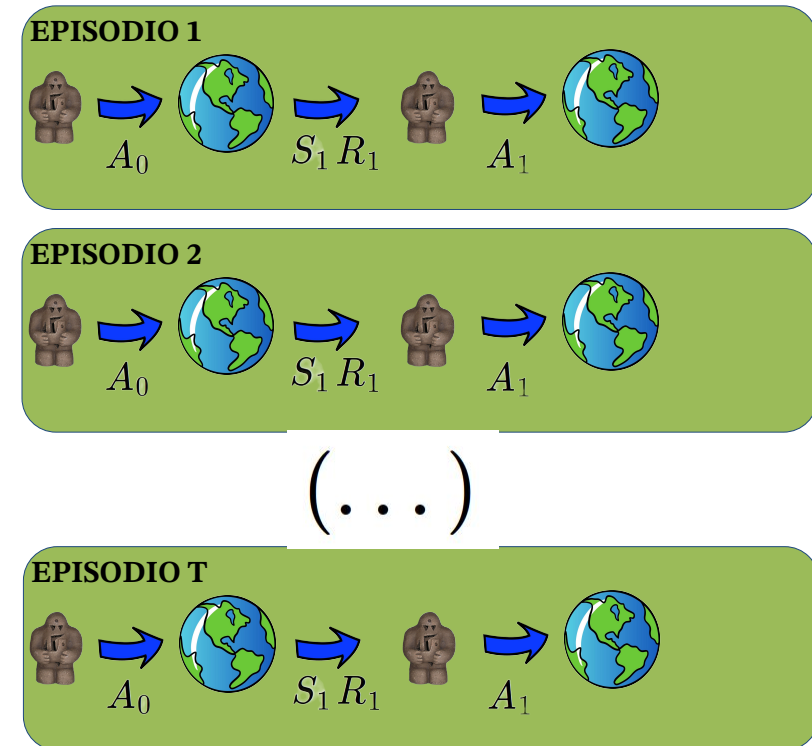
Algorithm 2: ϵ -greedy for bandit problems.

```
input  :  $\hat{Q}(a)$  arbitrarily initialized and learning  
        rates  $\lambda_t(a) \in (0, 1] \forall a \in \mathcal{A}$ ,  $\epsilon \in (0, 1]$   
for  $t$  in  $1 \dots T$  do  
    generate a random number  $j$   
    if  $j \leq \epsilon$  then  
        choose  $a$  at random  
    else  
        choose  $a = \arg \max_{a \in \mathcal{A}} \hat{Q}(a)$   
    observe  $r$   
    update  $\hat{Q}$ :  
         $\hat{Q}(a) \leftarrow \hat{Q}(a) + \lambda_t(a)[r - \hat{Q}(a)]$ 
```

Ejemplito

¿Cómo haríamos para programar y probar esta idea en un “entorno” de juguete?

- Definir los arms (*pdfs*)
- Definir el agente (la policy)
- Tener un loop que simule la interacción *Agente* <> *Arms*
- Evaluar cómo salió
- Juntar estadística



Definimos los arms

Definimos el agente

El loop que simula

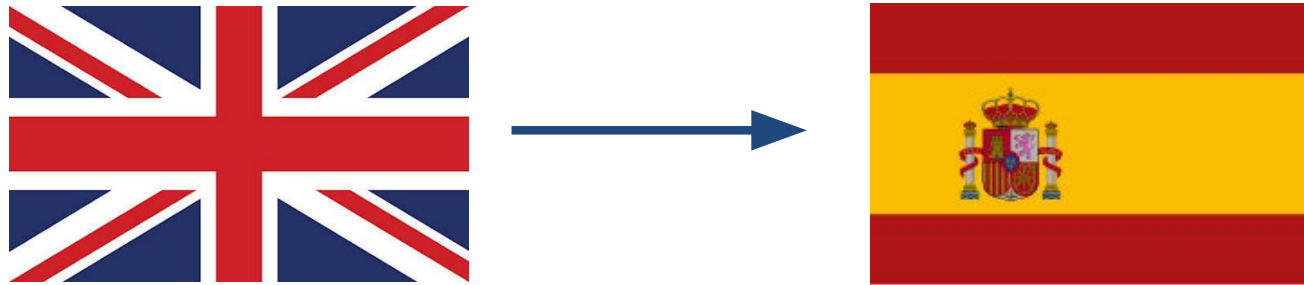
A ver qué pasa...

A ver qué pasa... (juntamos est.)

Elijo creer

¿Qué pasa con la inicialización?

“Optimism under the face of uncertainty”



Algorithm 2: ϵ -greedy for bandit problems.

input : $\hat{Q}(a)$ arbitrarily initialized and learning rates $\lambda_t(a) \in (0, 1] \forall a \in \mathcal{A}$, $\epsilon \in (0, 1]$
for t **in** $1 \dots T$ **do**
 generate a random number j
 if $j \leq \epsilon$ **then**
 choose a at random
 else
 choose $a = \arg \max_{a \in \mathcal{A}} \hat{Q}(a)$
 observe r
 update $\hat{Q}$:
 $\hat{Q}(a) \leftarrow \hat{Q}(a) + \lambda_t(a)[r - \hat{Q}(a)]$

Una idea: init $\hat{q}_1(a)$, $\forall a$ grande, y a medida que “decepciona” se van reduciendo solos, la estadística acumulada.

Upper Confidence Bounds

Desigualdad de Hoeffding. Sean $\{X_1 \dots X_n\}$ variables aleatorias independientes e idénticamente distribuidas (osea, sampleadas siempre del mismo arm que no cambia).

Sea $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$ y sea μ el valor medio ($\langle X \rangle$), entonces:

$$\Pr[\bar{X}_n + \epsilon \leq \mu] \leq e^{-2n\epsilon^2}$$

¿Cuál es la intuición de esto? “No te vas muy lejos del μ ”

Sobre qué es esta probabilidad???

Upper Confidence Bounds

¿Por qué nos sirve?

Si en lugar de decir “seamos optimistas” podemos acotar...

$$q(a) \leq \hat{q}_t(a) + U_t(a)$$

entonces podemos ser “optimistas” con criterio

$U_t(a)$ es la “upper confidence”...

¿Cómo esperamos que dependa “*confianza*” *c. experiencia*?

Upper Confidence Bounds

$$\Pr[\bar{X}_n + \epsilon \leq \mu] \leq e^{-2n\epsilon^2}$$

$$\Pr[\hat{q}_t(a) + U_t \leq q(a)] \leq e^{-2N_t(a)U_t^2}$$

Entonces, nos ponemos en modo “elijo creer” al UCB y...

Fijamos el valor para la probabilidad
del evento “saturo bound” como parámetro)
Luego *despejamos...*

$$U_t = \sqrt{\frac{\log 1/p}{2N_t(a)}}$$

Upper Confidence Bounds

Lo que hace UCB es seleccionar la acción según

$$a_t = \operatorname{argmax}_a (\hat{q}_t(a) + U_t(a)) \qquad U_t = \sqrt{\frac{\log 1/p}{2N_t(a)}}$$

En la práctica, se pone un *schedule* al parámetro $p \sim 1/t$ & c

$$U_t = c \sqrt{\frac{\log t}{N_t(a)}}$$

Algorithm 3: UCB for bandit problems.

```
input  :  $\mathcal{P}(t)$ , initialize  $\hat{Q}(a), N(a)$  to zero  $\forall a \in \mathcal{A}$ .  
for  $t$  in  $1, \dots, T$  do  
  if  $t \leq |\mathcal{A}|$  then  
    (choose each action once)  $a = t$   
  else  
    choose  $a = \arg \max_{a \in \mathcal{A}} ucb_t(a)$ ,  
    observe reward  $r$   
    record visit:  
       $N_t(a) \leftarrow N_t(a) + 1$   
    update Q-value:  
       $\hat{Q}(a) \leftarrow \hat{Q}(a) + \frac{[r - \hat{Q}(a)]}{N_t(a)}$ 
```

**Elijo creer → creer con
estructura**

Thompson Sampling

... de hecho la primera vez que se estudió Bandits (1933!)

Biometrika / Vol. 25, No. 3/4, Dec., 1933 / On the Likelihood that One Unknown Proba...

[Thompson 1933]

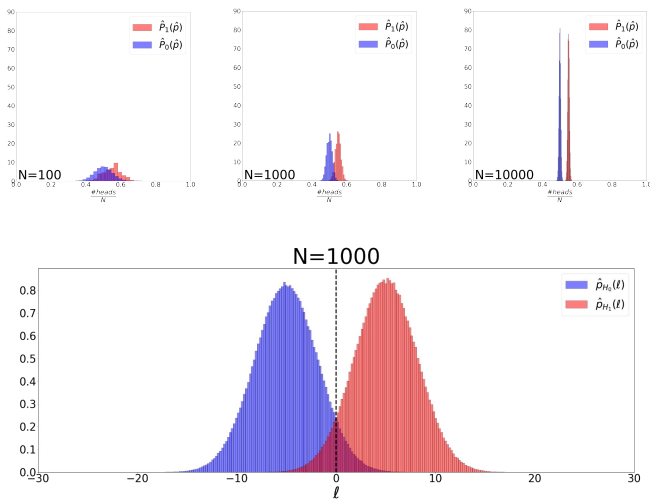
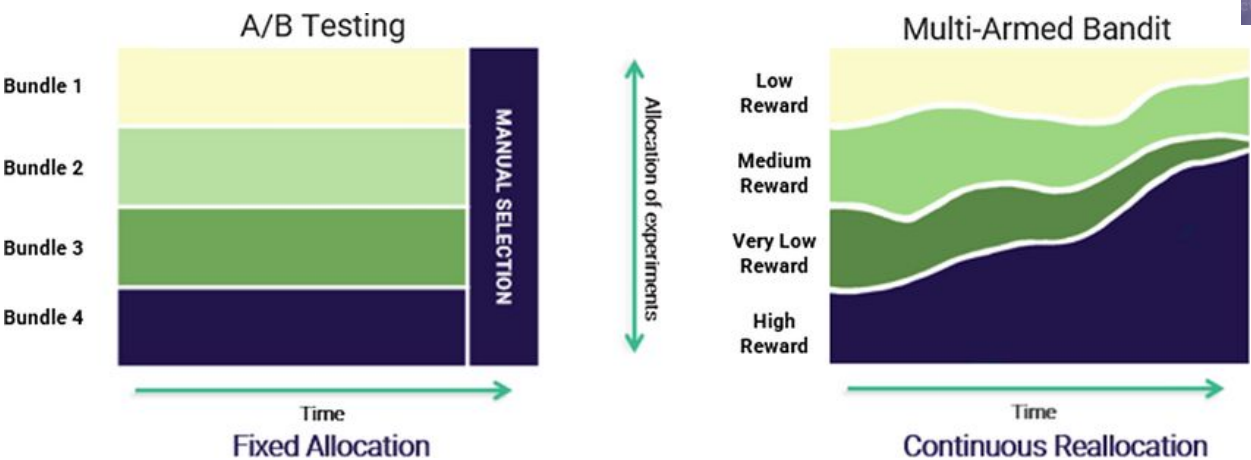
JOURNAL ARTICLE

On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples

William R. Thompson

Biometrika, Vol. 25, No. 3/4 (Dec., 1933), pp. 285-294 (10 pages)

<https://doi.org/10.2307/2332286> • <https://www.jstor.org/stable/2332286>

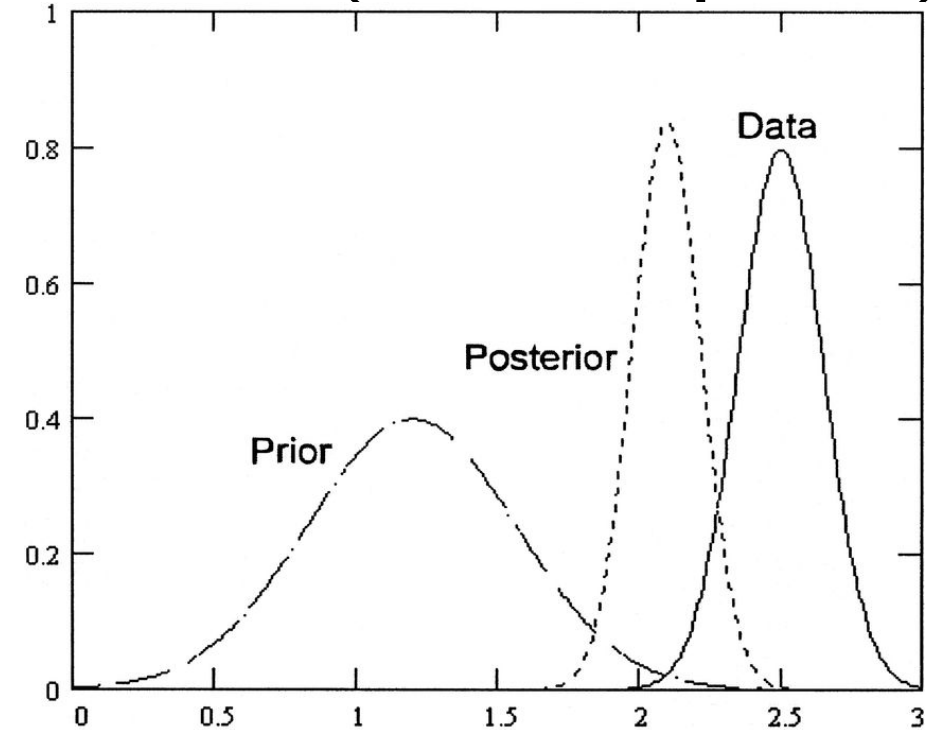


Guarda cierta relación con métodos deterministas / secuenciales en hypothesis testing. [SPRT-quantum]

¿Y si “modelamos” cada *arm*

Distribución “prior belief” → data → posterior (= new prior)

$$p(\hat{\theta}|R_t) = \frac{p_{\text{model}}(R_t|\hat{\theta})p_{\text{prior}}(\hat{\theta})}{p_{\text{total}}(R_t)}$$



- Mantenemos una creencia sobre cuán bueno es cada máquina^x
- (Fijate que podría ser un parámetro del modelo!) [\[Inferencia bayesiana\]](#)
- → Mantenemos creencia (parametrizada!) del reward medio q .
- Osea, para nosotros, $\theta \rightarrow q$ (pero esto es súper general!)

Thompson Sampling

Imagen (¿sirve?) gemelo digital (GD) de cada máquina / arm

- Convertimos (“internamente”, en el GD) a $\hat{q}(a)$ en V.A.
- Somos “greedy” en un juego interno (en el GD) $\leftarrow$ sample All
- Actualizamos el belief usando Bayes

$$p(\hat{\theta}|R_t) = \frac{p_{\text{model}}(R_t|\hat{\theta})p_{\text{prior}}(\hat{\theta})}{p_{\text{total}}(R_t)}$$



Actualizar el belief... “truco”



$$p(\hat{\theta}|R_t) = \frac{p_{\text{model}}(R_t|\hat{\theta})p_{\text{prior}}(\hat{\theta})}{p_{\text{total}}(R_t)}$$

**FAMILIAS
CONJUGADAS**

- p_{creo} “nuevo” ¿qué es?
- Evaluar p_{model} sobre cada θ
- Calcular p_{total}

- p_{creo} “nuevo” mantiene estructura
- Fórmula cerrada para el update
- eficiente (no evaluamos p_{model} sino implícitamente, en fórmula)

March, 1979

Conjugate Priors for Exponential Families

[Persi Diaconis](#), [Donald Ylvisaker](#)

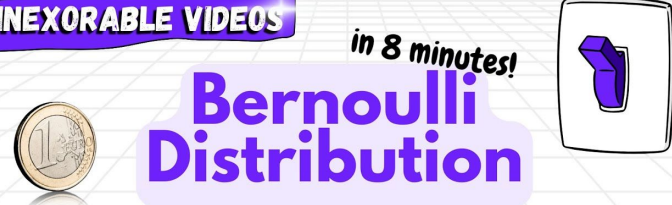
(Bernoulli, Poisson, Gauss c/ sigma known, exponencial)

El nuevo posterior “belief”/”creo” se obtiene en $O(1)$ (hay una fórmula)

Actualizar el belief... “truco”

INEXORABLE VIDEOS in 8 minutes!

Bernoulli Distribution



$$P(X = x) = \begin{cases} p & \text{if } x = 1 \\ 1 - p & \text{if } x = 0 \end{cases}$$

Beta Distribution

β $f(x) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha, \beta)}$



$$p(\hat{\theta}|R_t) = \frac{p_{\text{model}}(R_t|\hat{\theta})p_{\text{prior}}(\hat{\theta})}{p_{\text{total}}(R_t)}$$

- p_{creo} “nuevo” ¿qué es?
- Evaluar p_{model} sobre cada θ
- Calcular p_{total}

FAMILIAS CONJUGADAS

- p_{creo} “nuevo” mantiene estructura
- Fórmula cerrada para el update
- eficiente (no evaluamos p_{model} sino implícitamente, en fórmula)

March, 1979
Conjugate Priors for Exponential Families
[Persi Diaconis](#), [Donald Ylvisaker](#)
(Bernoulli, Poisson, Gauss σ known, exponencial)

El nuevo posterior “belief”/”creo” se obtiene en $O(1)$ (hay una fórmula)

$$p_{\text{prior}}(\hat{q}_a; \alpha, \beta) \propto q_a^{\alpha-1}(1-q_a)^{1-\beta}$$

$$p_{\text{model}}(R_t|q_a) = \text{Ber}(R_t; q_a)$$

$$p_{\text{creo}}(q_a) = \text{Beta}(q_a; \alpha + R_t, \beta + 1 - R_t) \propto q_a^{\alpha+R_t-1}(1-q_a)^{1-(\beta+1-R_t)}$$

➡ $p(\hat{\theta}|R_t) = \frac{p_{\text{model}}(R_t|\hat{\theta})p_{\text{prior}}(\hat{\theta})}{p_{\text{total}}(R_t)}$

Se reduce a actualizar los parámetros α, β

Thompson Sampling

Intuitivo: gemelo digital de cada máquina / arm

Algorithm 4: TS for Bernoulli bandit problems.

input : $\mu_1(a), \nu_1(a)$ initialized to one $\forall a \in \mathcal{A}$

for t **in** $1, \dots, T$ **do**

for a **in** $\mathcal{A}$ **do**

 draw $\bar{r}_a$ according to $\text{Beta}(\mu_t(a), \nu_t(a))$

 choose $a = \arg \max_a \bar{r}_a$

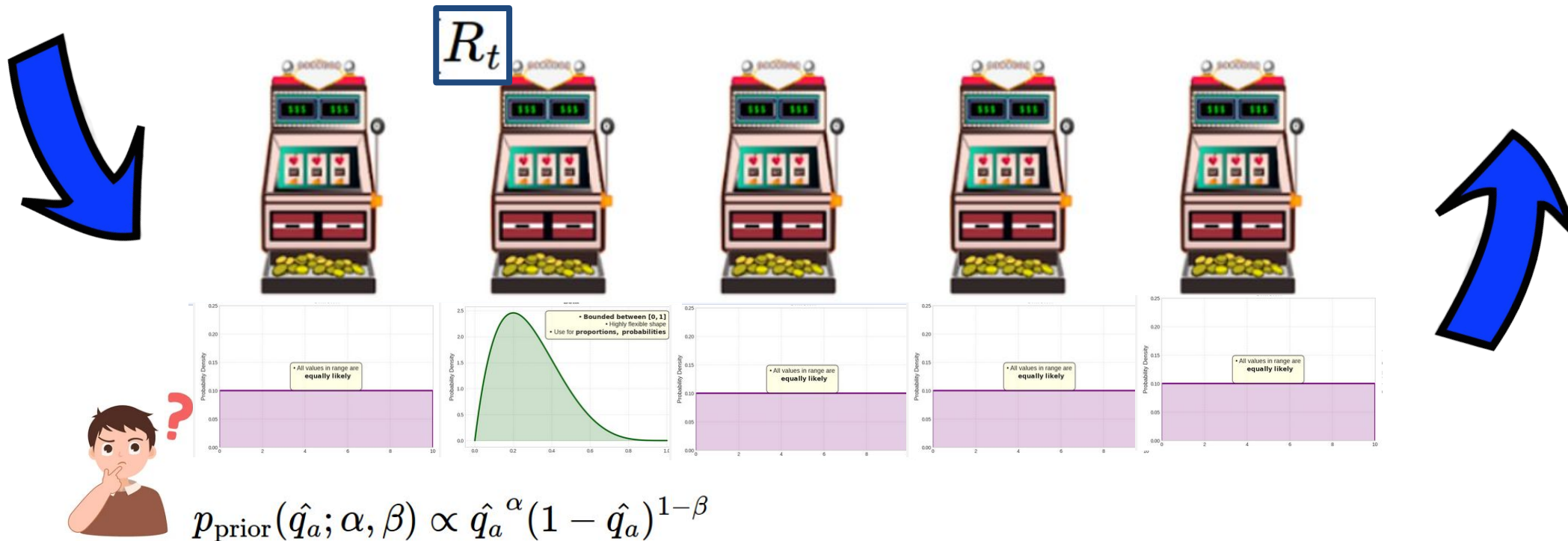
 observe reward r

 update Beta distribution:

$\mu_{t+1}(a) = \mu_t(a) + r$

$\nu_{t+1}(a) = \nu_t(a) + 1 - r$

$$p_{\text{creo}}(q_a) = \text{Beta}(q_a; \alpha + R_t, \beta + 1 - R_t) \propto q_a^{\alpha + R_t - 1} (1 - q_a)^{1 - (\beta + 1 - R_t)}$$



Thompson Sampling

Algorithm 4: TS for Bernoulli bandit problems.

input : $\mu_1(a), \nu_1(a)$ initialized to one $\forall a \in \mathcal{A}$

for t *in* $1, \dots, T$ **do**

for a *in* $\mathcal{A}$ **do**

 draw $\bar{r}_a$ according to $\text{Beta}(\mu_t(a), \nu_t(a))$

 choose $a = \arg \max_a \bar{r}_a$

 observe reward r

 update Beta distribution:

$$\mu_{t+1}(a) = \mu_t(a) + r$$

$$\nu_{t+1}(a) = \nu_t(a) + 1 - r$$

Lo mismo vale para Gaussianas



$$p(R_t|\mu) = \mathcal{N}(\mu, \sigma^2)$$

$$p_{\text{prior}}(\mu) = \mathcal{N}(\mu, \sigma^2)$$



$$p(\hat{\theta}|R_t) = \frac{p_{\text{model}}(R_t|\hat{\theta})p_{\text{prior}}(\hat{\theta})}{p_{\text{total}}(R_t)}$$

- p_{creo} “nuevo” ¿qué es?
- Evaluar p_{model} sobre cada θ
- Calcular p_{total}

FAMILIAS
CONJUGADAS

- p_{creo} “nuevo” mantiene estructura
- Fórmula cerrada para el update
- eficiente (no evaluamos p_{model} sino implícitamente, en fórmula)

March, 1979
Conjugate Priors for Exponential Families
[Persi Diaconis](#), [Donald Ylvisaker](#)
(Bernoulli, Poisson, Gauss c/ sigma known, exponencial)

El nuevo postirior “belief”/”creo” se obtiene en O(1) (hay una fórmula)

$$m_{\text{new}} = \tau_{\text{new}}^2 \left(\frac{m_{\text{old}}}{\tau_{\text{old}}^2} + \frac{r}{\sigma^2} \right)$$
$$\tau_{\text{new}}^2 = \left(\frac{1}{\tau_{\text{old}}^2} + \frac{1}{\sigma^2} \right)^{-1}$$



Thompson Sampling

Algunas limitantes:

- no siempre el bayesian update tiene una forma cerrada
(o el likelihood que usamos para los R_t está disponible)
(o el likelihood que usamos para los R_t está mal)*
- Necesitamos un prior por acción + hacer BU (escala fatal!)
* ¿y si no es estacionario?*
- Pero es interesantísimo y se usa en un montón de problemas
Más biblio → [\[A tutorial on Thompson Sampling\]](#)*

Figuras de mérito

¿Qué vimos hasta ahora?

De momento dijimos que nos interesa maximizar $\bar{R} = \sum_i R_i$ y que esto es una variable aleatoria (modelo + “policy”)

*¿¿Podemos estudiar la distribución de probabilidad??
O al menos decir algo interesante ? $p(\bar{R}; \pi, \vec{q}, \dots)$*

Mensaje hay mucha tela para cortar desde la parte teórica.
En RL no nos vamos a concentrar (en este curso) en esto
(mi perspectiva, quizá errónea, es que en la práctica es
difícil evaluar cuestiones...)

Regret | arrepentimiento

*“No hay nostalgia peor //
que añorar lo que nunca jamás
sucedió...”*

[Con la frente marchita, Sabina]

Def. Gap $\Delta_{A_t} := q_* - q(A_t)$ (¿está claro que $\text{gap} > 0$?)

Def. Simple regret $\ell_t = \mathbb{E}[\Delta_{A_t}]$ ¿Por qué el E ?

donde $$ es el mejor “arm”*

Observación: si pensamos en que el objetivo sea optimizar el simple regret, el juego cambia completamente; es más bien un problema de “best-arm” identification (sin costo de explorar)

Regret | arrepentimiento

*“No hay nostalgia peor //
que añorar lo que nunca jamás
sucedió...”*

[Con la frente marchita, Sabina]

Def. *Cumulative regret*

$$L_T = \mathbb{E}\left[\sum_{t=1}^T \Delta_{A_t}\right] = \sum_a \mathbb{E}[N_T(a)] \Delta_a$$

*N_T es la cantidad de veces que elegimos “ a ” hasta episodio T
Por ende es una variable aleatoria (depende de a policy!)*

primera suma $\rightarrow$ tiempo,

segunda suma $\rightarrow$ acciones (te animás a demostrar

La cota de Lai-Robbins

Teorema Lai-Robbins (1985)

[\[Lai Robbins Bound\]](#)

$$L_T \geq \log T \sum_{a \neq *} \frac{\Delta_a}{\text{KL}(p_a(R) || p_*(R))}$$

$$KL(p|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \quad \text{o} \quad \int p(x) \log \frac{p(x)}{q(x)}, dx$$

¿Qué comportamiento esperamos para epsilon-greedy?

Resultados:

para la familia indicada, TS satura el bound
UCB tiene, en general, el scaling óptimo

Tarea

Leer S&B Capítul 2 (bandits), saltar la parte de gradient

Chusmear aplicaciones de Bandits (para darse una idea!)

- Sistema de recomendación de Netflix
- Multi-armed bandits meet LLMs

(Procedimiento, lo mismo que la clase pasada!)